

# CubeSat ADCS Hardware-in-the-Loop Design and Validation

Caleb Oblepias

## Contents

|          |                                     |           |
|----------|-------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                 | <b>2</b>  |
| <b>2</b> | <b>Physical Layer</b>               | <b>3</b>  |
| 2.1      | Overview . . . . .                  | 3         |
| 2.2      | Timing Diagram . . . . .            | 3         |
| 2.3      | Finite State Machine . . . . .      | 3         |
| 2.4      | Validation . . . . .                | 4         |
| <b>3</b> | <b>Protocol Layer</b>               | <b>5</b>  |
| 3.1      | Overview . . . . .                  | 5         |
| 3.2      | Commands . . . . .                  | 6         |
| 3.3      | Inputs and Outputs . . . . .        | 6         |
| 3.4      | Finite State Machine . . . . .      | 7         |
| 3.5      | Validation . . . . .                | 7         |
| <b>4</b> | <b>Register Layer</b>               | <b>9</b>  |
| 4.1      | Overview . . . . .                  | 9         |
| 4.2      | Register Map . . . . .              | 9         |
| 4.3      | Validation . . . . .                | 10        |
| <b>5</b> | <b>Control FSM</b>                  | <b>11</b> |
| 5.1      | Overview . . . . .                  | 11        |
| 5.2      | Finite State Machine . . . . .      | 11        |
| 5.3      | Validation . . . . .                | 11        |
| <b>6</b> | <b>Numerical Integration Module</b> | <b>12</b> |
| 6.1      | Overview . . . . .                  | 12        |
| 6.2      | Operation . . . . .                 | 12        |
| 6.3      | Finite State Machine . . . . .      | 13        |
| 6.4      | Validation . . . . .                | 13        |
| <b>7</b> | <b>Next Steps</b>                   | <b>15</b> |
| <b>8</b> | <b>Significance</b>                 | <b>16</b> |

## Introduction

This report presents the design and implementation of a layered SPI-based communication interface enabling hardware-in-the-loop (HIL) interaction between an STM32 microcontroller and an FPGA-based simulation platform. The system is architected to separate concerns across well-defined abstraction layers, allowing precise control over timing at the physical level while maintaining flexibility and modularity at higher levels of the design. This layered approach not only simplifies development and debugging but also enables independent verification of each subsystem.

At the lowest level, the SPI physical layer interfaces directly with the serial communication signals, handling bit-level timing, serialization, and deserialization in accordance with SPI mode 0 operation. Above this, the protocol layer interprets byte-level transactions, decoding commands and orchestrating read and write operations. These operations are mapped onto a centralized register map, which serves as the interface boundary between communication logic and application logic. The register map exposes control and state variables to both the embedded controller and the FPGA's internal modules.

On the FPGA side, the register map connects to higher-level functional blocks, including a control finite state machine and a numerical integration module implementing rigid body dynamics via RK4 or Euler methods. This structure enables the STM32 to issue commands, configure system parameters, and retrieve simulation results in real time. By decoupling communication, control, and computation, the architecture provides a scalable framework for HIL experimentation, where deterministic timing, low-latency data exchange, and modular extensibility are critical design goals.

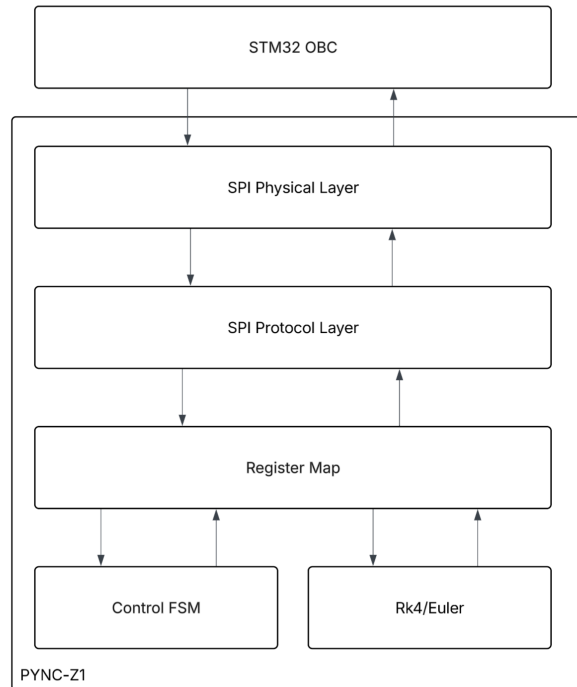


Figure 1: System Architecture

## Physical Layer

### Overview

The physical layer implements the direct electrical and timing interface between the SPI master and the internal logic of the design. It operates at the granularity of individual clock edges, handling the serialization and deserialization of data on the MOSI and MISO lines. Incoming data is sampled on the rising edge of the serial clock (SCLK), consistent with SPI mode 0 operation, and shifted into an internal register until a full byte is assembled. A small counter tracks the number of received bits, and once eight bits have been captured, the byte is presented to the protocol layer alongside a one-cycle “done” pulse indicating valid data. This mechanism cleanly decouples bit-level timing from byte-level processing, allowing higher layers to operate without concern for individual clock edges.

The transmit path is implemented on the falling edge of SCLK to ensure proper setup time for the master’s sampling edge. A separate shift register drives the MISO line, outputting one bit per clock cycle. At byte boundaries, this register is reloaded with the latest transmit data provided by the protocol layer, ensuring that each outgoing byte is aligned with the SPI clock. Importantly, the transmit logic is self-contained and does not rely on signals generated in the input (posedge) domain, avoiding cross-edge timing hazards. Chip select (CS) acts as a transaction-level reset, reinitializing counters and shift registers to guarantee proper alignment at the start of each transfer. Together, these mechanisms provide a deterministic and timing-correct physical interface upon which the higher-level SPI protocol is built.

### Timing Diagram

The SPI interface in this design operates in Mode 0 (CPOL = 0, CPHA = 0), meaning the clock idles low and data is sampled on the rising edge of SCLK. Accordingly, input data on MOSI is captured on the rising edge, while output data on MISO is updated on the falling edge to ensure proper setup time before the next sampling event.

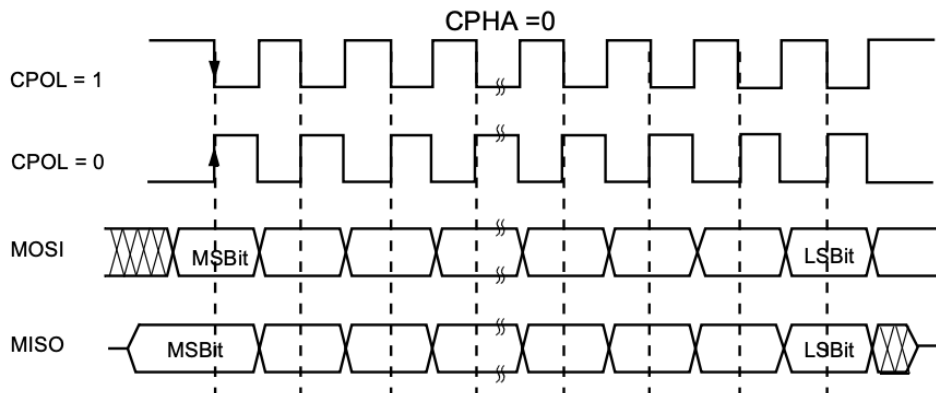


Figure 2: SPI timing diagram

### Finite State Machine

The physical layer finite state machine governs byte-level reception by tracking the progression of incoming bits under the control of the serial clock and chip select signals. When CS is asserted, the system remains in an idle state; once deasserted, it transitions to a receiving state where bits

are shifted in on each rising clock edge until a full byte is assembled. Upon receiving eight bits, the FSM enters a done state and generates a one-cycle valid signal before returning to idle, ensuring clean byte delineation and synchronization with the protocol layer.

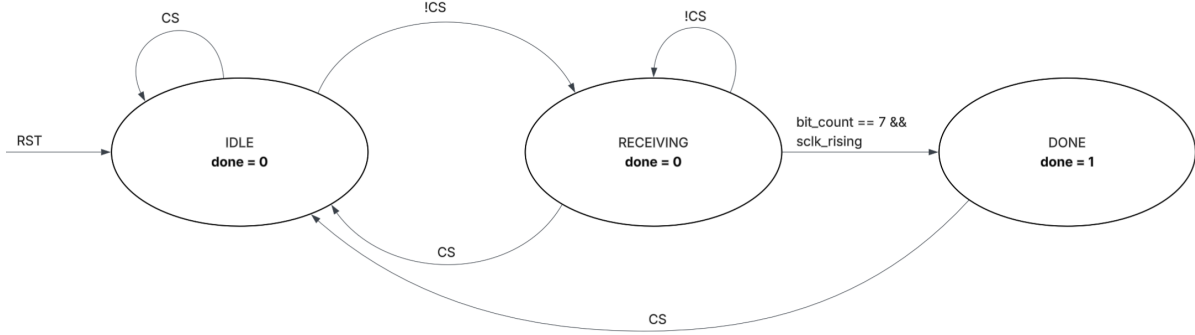


Figure 3: Physical FSM

## Validation

To validate the behavior of the SPI interface, an initial test was performed using a fixed, known transmit value on the FPGA. The STM32 was configured to continuously send a constant byte (0x07) while the FPGA was hardcoded to return a known value (0xAA) on the MISO line. This setup allowed for straightforward verification of correct physical layer timing, byte alignment, and protocol handling without the added complexity of dynamic data dependencies. By observing the returned data on the STM32 side, it was confirmed that the SPI slave correctly serialized and transmitted the expected value at the appropriate byte boundary, validating both the transmit path and the overall timing relationship between SCLK edges and data movement.

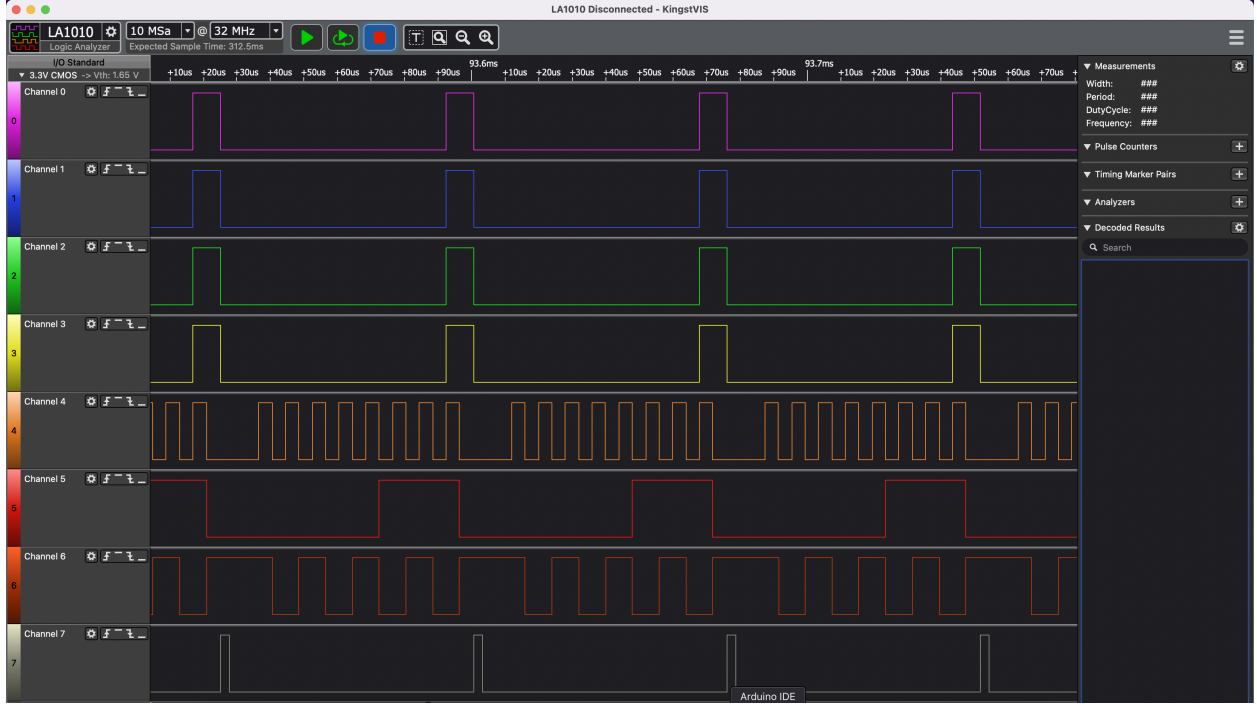


Figure 4: SPI validation, reading 0x07 and writing 0xAA

Following this, a second validation step introduced dynamic behavior by modifying the FPGA to return an incrementing value on each read transaction, starting from 0x01. This test verified that the full data path, from register update to protocol handling to physical transmission, was functioning correctly. By confirming that the STM32 received a monotonically increasing sequence of values across successive transactions, it was demonstrated that the system correctly handled state progression, data updates, and synchronization across multiple SPI frames. This provided confidence that the interface supports real-time data exchange and is suitable for integration with higher-level modules such as the control FSM and numerical simulation blocks.

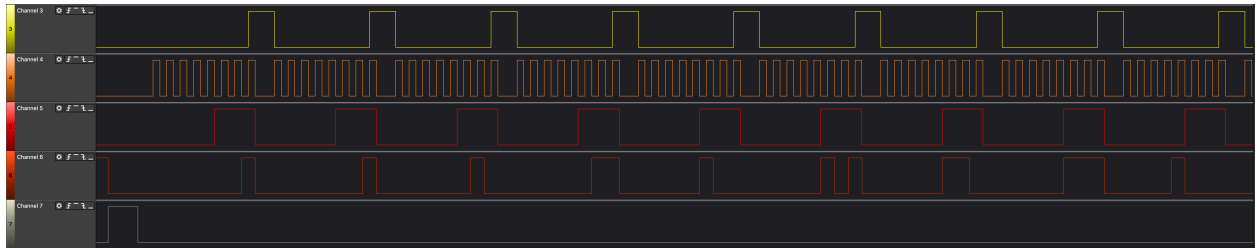


Figure 5: SPI incrementing write validation

## Protocol Layer

### Overview

The protocol layer operates above the physical SPI interface and is responsible for interpreting byte-level transactions into meaningful read and write operations on the system. While the physical layer handles bit-accurate timing and serialization, the protocol layer abstracts this into discrete

bytes using a finite state machine (FSM). This FSM sequences through command, address, and data phases, allowing the system to distinguish between different transaction types. In this design, a simple command structure is used, where a command byte determines whether the operation is a read or write, followed by an address byte and, in the case of writes, a data byte. This structured approach enables deterministic parsing of incoming data and ensures that each transaction is handled in a predictable and repeatable manner.

For read operations, the protocol layer coordinates with the register map to retrieve the requested data while accounting for inherent system latency. Upon receiving an address, the corresponding register value is accessed and prepared for transmission back to the master. Due to the pipelined nature of the SPI interface, the protocol layer ensures that transmit data is staged early enough to align with the physical layer's byte boundaries, avoiding timing hazards. For write operations, the protocol asserts a write-enable signal and updates the specified register with the incoming data. By cleanly separating transaction decoding, control signaling, and data movement, the protocol layer provides a robust bridge between the serial communication interface and the internal system logic, enabling reliable and extensible communication.

## Commands

- 0x01: Write
- 0x02: Read

Transaction format:

- Write: [CMD][ADDR][DATA]
- Read: [CMD][ADDR] → [DATA]

The SPI protocol supports two primary commands: a write command (0x01) and a read command (0x02). A write transaction consists of three sequential bytes, command, address, and data, allowing the embedded controller to update a register within the FPGA. A read transaction consists of a command and address phase, followed by additional clock cycles during which the FPGA returns the requested data. This simple command structure minimizes protocol overhead while maintaining flexibility for accessing all registers in the system.

## Inputs and Outputs

The protocol layer interfaces with the SPI physical layer through a set of well-defined signals. Inputs such as `sclk`, `cs`, `rx_data`, and `rx_valid` provide timing and received data information, while `rdata` supplies data from the internal register map during read operations. Outputs including `we`, `waddr`, and `wdata` control write operations to the register map, while `raddr` specifies the address for read access. The `tx_data` signal carries data back to the physical layer for transmission over MISO. This separation enables a modular design between communication and computation.

| Inputs                | Outputs              |
|-----------------------|----------------------|
| <code>sclk</code>     | <code>we</code>      |
| <code>cs</code>       | <code>waddr</code>   |
| <code>rx_data</code>  | <code>wdata</code>   |
| <code>rx_valid</code> | <code>raddr</code>   |
| <code>rdata</code>    | <code>tx_data</code> |

Table 1: SPI Protocol Interface Signals

## Finite State Machine

The protocol behavior is governed by a finite state machine that sequences command decoding and data movement. The system begins in a WAIT\_CMD state, where it captures the incoming command byte, then transitions to WAIT\_ADDR to receive the target register address. Depending on the command, the FSM either proceeds to WAIT\_DATA for write operations or transitions through READ\_DATA and LOAD\_DATA states for read operations. This staged approach ensures proper timing alignment between received bytes and outgoing data, accounting for the latency required to fetch register values. The FSM provides deterministic behavior and maintains synchronization with the SPI byte-level interface.

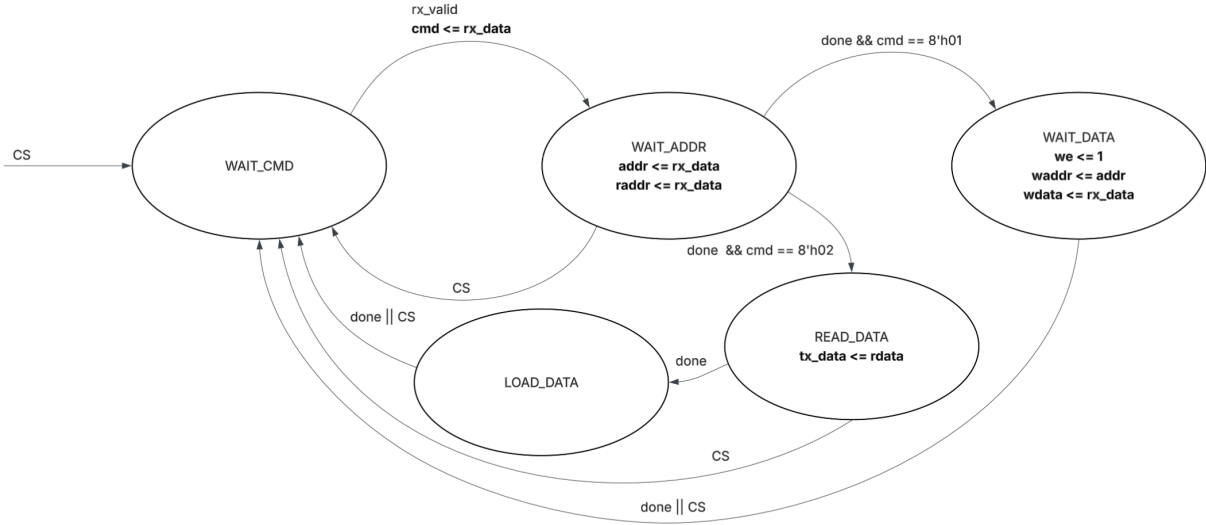


Figure 6: Protocol FSM

## Validation

The write transaction is validated by issuing a command to store the value 0xA7 at register address 0x05. The STM32 transmits the command byte 0x01, followed by the address byte 0x05, and finally the data byte 0xA7. During this process, the chip select signal remains asserted and the serial clock drives the transfer of each bit over the MOSI line. Upon reception of the final byte, the protocol layer asserts the write enable signal and updates the register map accordingly. The observed waveform confirms correct command decoding, address capture, and data write timing, indicating that the protocol layer successfully performs the intended write operation.

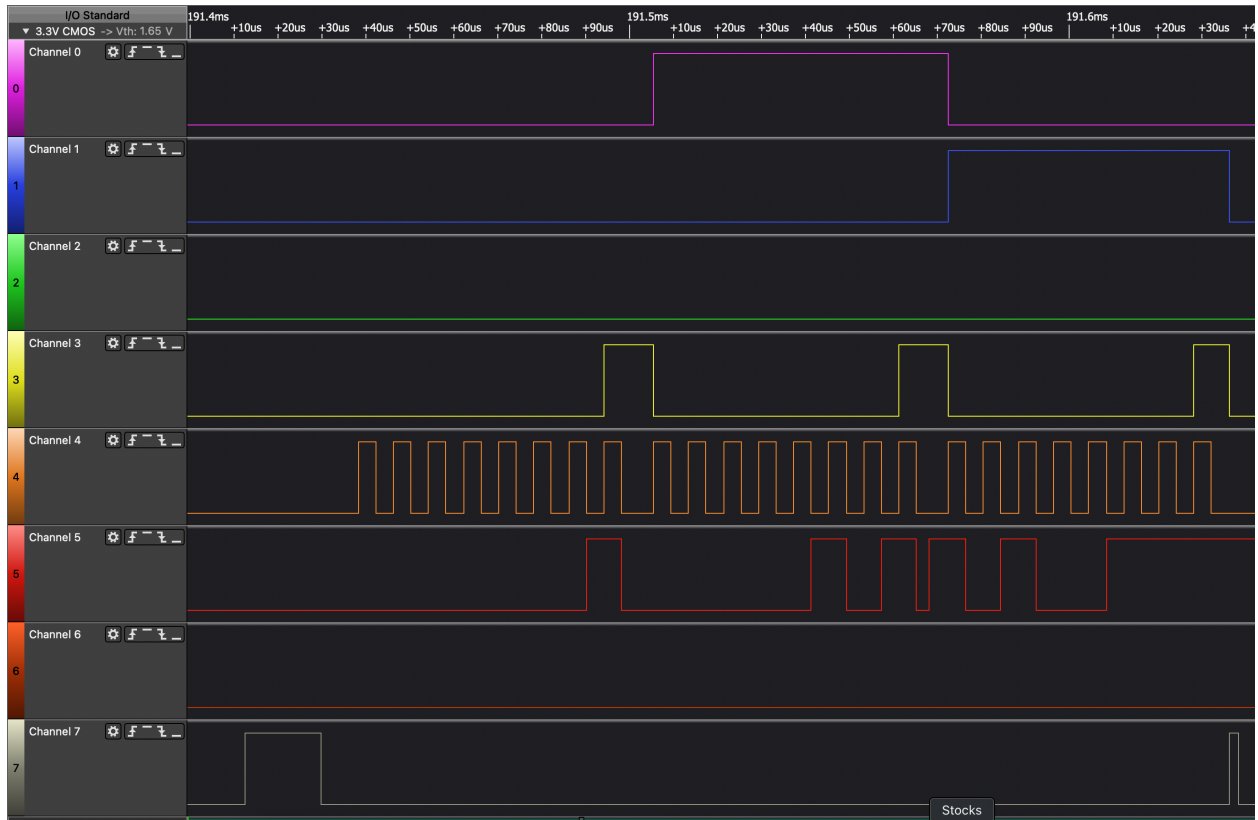


Figure 7: Protocol Write

The read transaction is validated by requesting the value stored at register address `0x05`. The STM32 transmits the read command `0x02`, followed by the address byte `0x05`, and then continues clocking additional cycles to receive data. During these cycles, the protocol layer retrieves the stored value `0xA7` from the register map and presents it on the transmit path. The returned data is observed on the MISO line, confirming that the correct value is shifted out in response to the read request. This demonstrates proper synchronization between the receive and transmit paths and verifies that the protocol correctly returns previously written data.



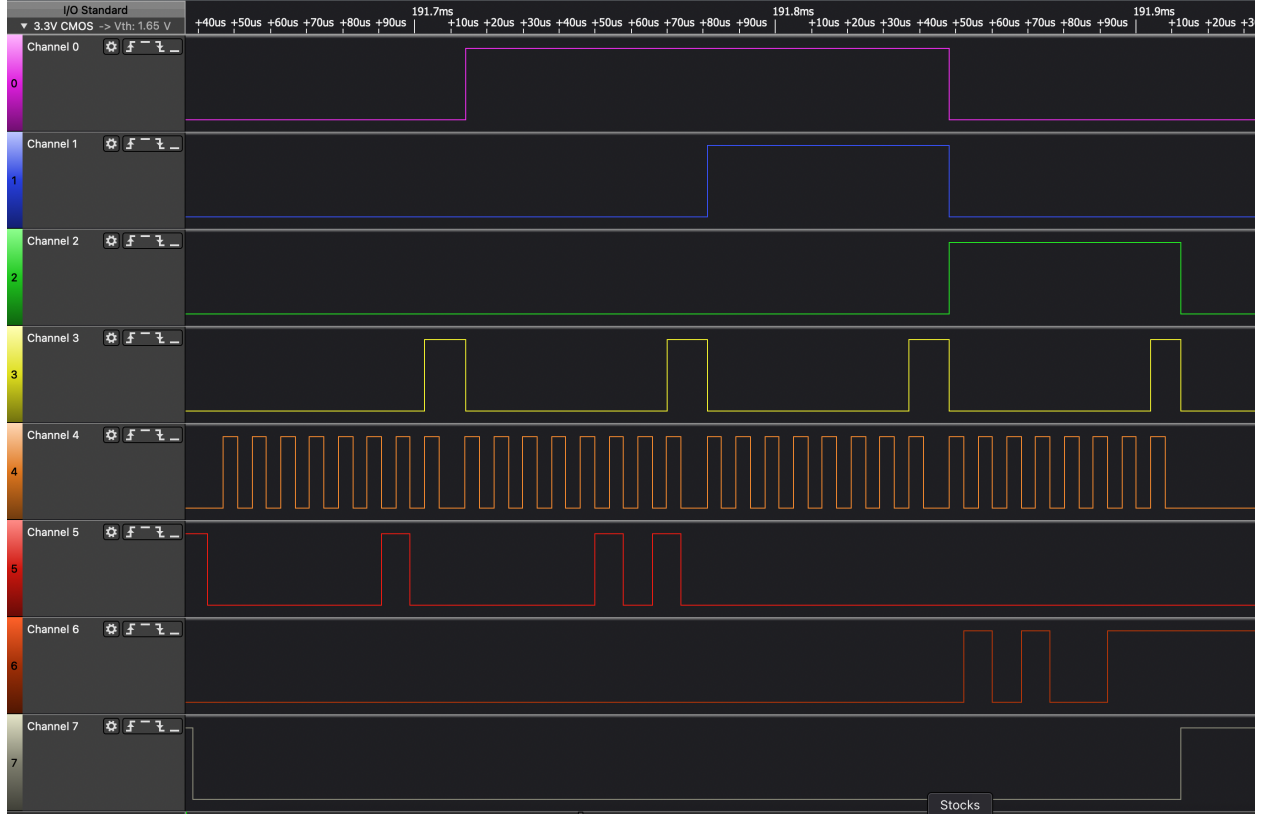


Figure 8: Protocol Read

## Register Layer

### Overview

The register layer serves as the interface between the SPI communication stack and the internal simulation logic. It provides a structured memory-mapped abstraction that allows the embedded controller to interact with the FPGA system using simple read and write transactions. By decoupling communication from computation, this layer enables clean modularity, allowing the SPI protocol to remain generic while the underlying system behavior is implemented independently. All external interaction with the simulation, including initialization, control, and observation of system states, is performed through this register interface.

In addition to enabling communication, the register layer enforces deterministic data flow between the controller and the simulation modules. Inputs from the controller, such as initial angular velocities or applied torques, are written into designated registers and consumed by the internal computation blocks. Similarly, computed outputs such as angular velocity states are exposed through read-accessible registers. This structure ensures that data exchange is synchronized to discrete transactions, avoiding ambiguity in timing and allowing the system to operate reliably within the hardware-in-the-loop framework.

### Register Map

The register map defines the addressable interface through which the controller interacts with the system. Control and configuration registers, such as `CONTROL` and `STATUS`, provide system-level

coordination and feedback, while input registers such as `INIT_WX`, `INIT_WY`, and `INIT_WZ` are used to initialize the state of the simulation. Torque inputs are provided through `TORQUE_X`, `TORQUE_Y`, and `TORQUE_Z`, enabling external influence on the system dynamics. Output registers, including `WX_OUT`, `WY_OUT`, and `WZ_OUT`, expose the computed state of the system to the controller. Unused address locations are reserved for future expansion, ensuring scalability of the interface without requiring protocol modifications.

| Register | Number |
|----------|--------|
| STATUS   | 0x00   |
| CONTROL  | 0x01   |
| TORQUE_X | 0x02   |
| TORQUE_Y | 0x03   |
| TORQUE_Z | 0x04   |
| INIT_WX  | 0x05   |
| INIT_WY  | 0x06   |
| INIT_WZ  | 0x07   |
| –        | 0x08   |
| –        | 0x09   |
| WX_OUT   | 0x0A   |
| WY_OUT   | 0x0B   |
| WZ_OUT   | 0x0C   |
| –        | 0x0D   |
| –        | 0x0E   |
| –        | 0x0F   |

Table 2: Register Map

The register layer operates on a simple set of control signals that define read and write behavior. Inputs include the address, write data, and write enable signal, which together determine when and where data is stored within the register map. These signals are driven by the SPI protocol layer and correspond directly to decoded transaction fields. On the output side, the register layer provides read data based on the requested address, allowing the controller to retrieve system state information. Additional status signals may be exposed to indicate conditions such as computation completion or system readiness, supporting higher-level control logic.

## Validation

Validation of the register layer focuses on verifying correct data storage, retrieval, and address decoding. This is performed by issuing controlled write operations to specific register addresses and subsequently reading back the stored values to confirm integrity. For example, known values can be written to initialization and torque registers and then read back to ensure correct memory behavior. In addition, dynamic signals from the simulation, such as computed angular velocities, are monitored through the output registers to confirm that internal data is correctly propagated to the interface. Waveform analysis using a logic analyzer is used to correlate SPI transactions with register updates, ensuring proper synchronization between communication and internal state updates.

## Control FSM

### Overview

The control finite state machine (FSM) coordinates the execution of the simulation by managing initialization, computation triggering, and result availability. It operates on top of the register layer, using control signals written by the STM32 to determine when to initialize the system, begin computation, and indicate completion. This FSM ensures that the internal simulation modules are driven in a controlled and deterministic manner, preventing race conditions between configuration updates and computation cycles.

The FSM progresses through a sequence of states including initialization, idle, busy, and ready. During initialization, state variables are loaded from the register map, allowing the controller to define the starting conditions of the system. In the idle state, the FSM waits for a start command, ensuring that computation only begins when explicitly triggered. Once started, the system transitions to a busy state where the numerical integration module is executed. Upon completion, the FSM enters a ready state, signaling that valid output data is available for readback. This structure provides a clear separation between configuration, execution, and data retrieval phases.

### Finite State Machine

The control FSM transitions between INIT, IDLE, BUSY, and READY states based on control register inputs and internal completion signals. The INIT state loads initial angular velocity values from the register map, after which the system transitions to IDLE. From IDLE, a start signal triggers entry into the BUSY state, where torque inputs and current state values are passed into the numerical integration module. Once computation is complete, indicated by a done signal, the FSM updates the output registers and transitions to the READY state. The system remains in READY until a new computation cycle is requested or reinitialization is triggered.

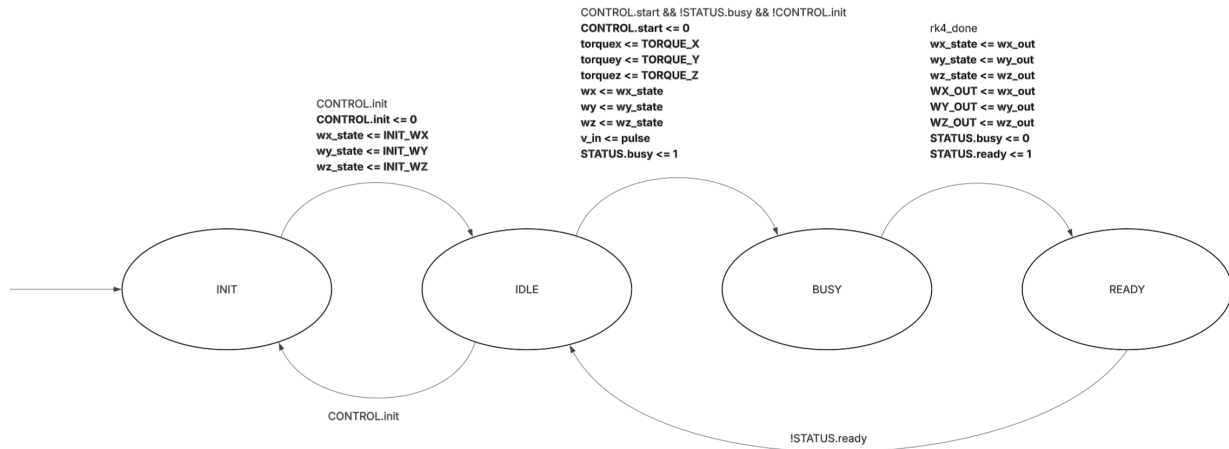


Figure 9: Control FSM

### Validation

Validation of the control FSM focuses on verifying correct sequencing of state transitions and proper coordination with the register layer and computation module. This is performed by issuing control

commands through the SPI interface, including initialization and start signals, and observing the resulting state progression. Specifically, the system is initialized with known values, a computation is triggered, and the status register is monitored to confirm transitions from IDLE to BUSY to READY. Additionally, output registers are checked after completion to ensure that results are only updated once the computation has finished. Logic analyzer waveforms are used to correlate control signals, state transitions, and output validity, confirming that the FSM correctly governs the execution flow of the system.

## Numerical Integration Module

### Overview

The numerical integration module implements the rigid body rotational dynamics of the system using a pipelined hardware architecture. It computes angular acceleration based on the current angular velocity, applied torques, and inertia parameters, following Euler's equations of motion. The design leverages parallel arithmetic operations and staged registers to achieve deterministic timing and high throughput, making it well-suited for hardware-in-the-loop applications where consistent latency is critical.

The module is structured as a multi-stage pipeline, where each stage performs a portion of the computation. Intermediate values such as inertia-weighted angular velocity and cross products are computed and stored across clock cycles, allowing the design to balance resource usage and performance. A valid signal propagates through the pipeline alongside the data, ensuring proper alignment between inputs and outputs. This approach enables continuous streaming of input data while maintaining correct synchronization across all computation stages.

### Operation

The computation begins by scaling angular velocity components by their corresponding inertia values, followed by a cross product operation that captures the nonlinear coupling between rotational axes. The resulting values are subtracted from the applied torque inputs to produce intermediate acceleration terms. These terms are then multiplied by the inverse inertia values to produce the final angular acceleration outputs. Each stage is registered, creating a fully pipelined data path with fixed latency from input to output.

## Finite State Machine

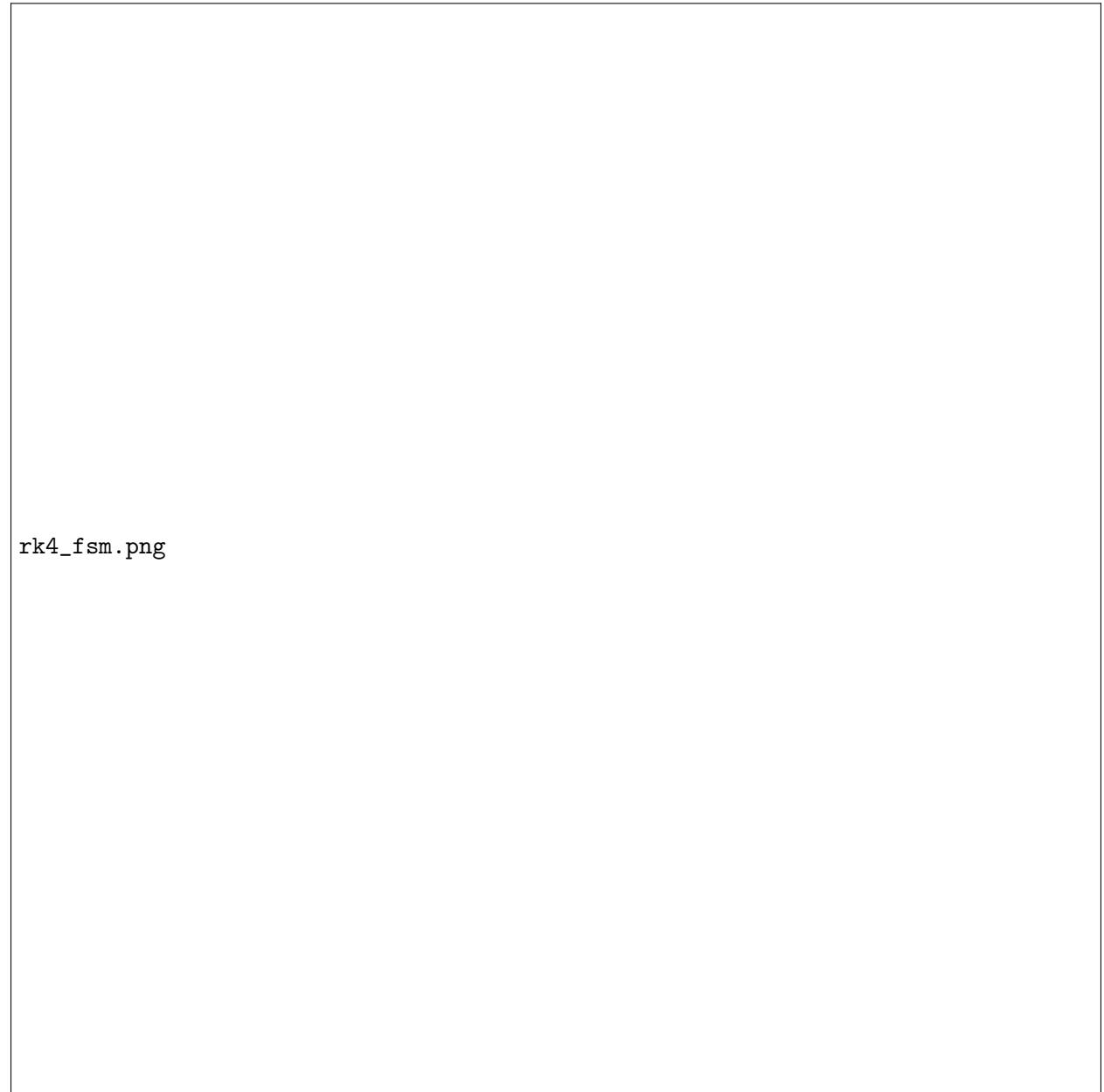


Figure 10: Numerical Integration FSM

## Validation

The first stage of validation focuses on verifying the correctness of the implemented Euler rotational dynamics. In this test, known values for inertia, angular velocity, and applied torque are provided as inputs, and the resulting angular accelerations are observed. The waveform confirms that intermediate pipeline signals such as the inertia-weighted angular velocity, cross product terms, and torque differences evolve as expected across clock cycles. Additionally, the final outputs for  $\dot{\omega}_x$ ,  $\dot{\omega}_y$ , and  $\dot{\omega}_z$  match analytically computed results, demonstrating that the hardware implementation

correctly evaluates Euler’s equations of motion. The propagation of the valid signal through the pipeline further confirms proper synchronization between data and control paths.

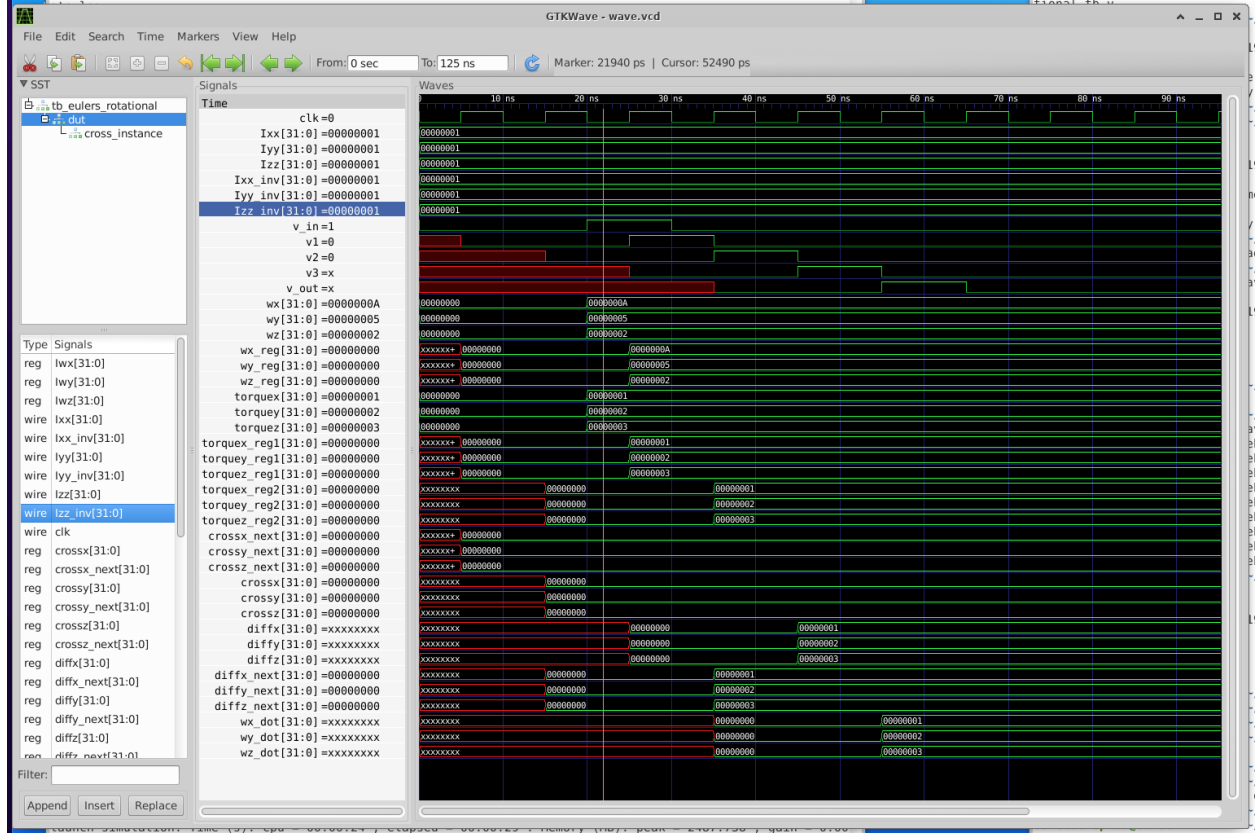
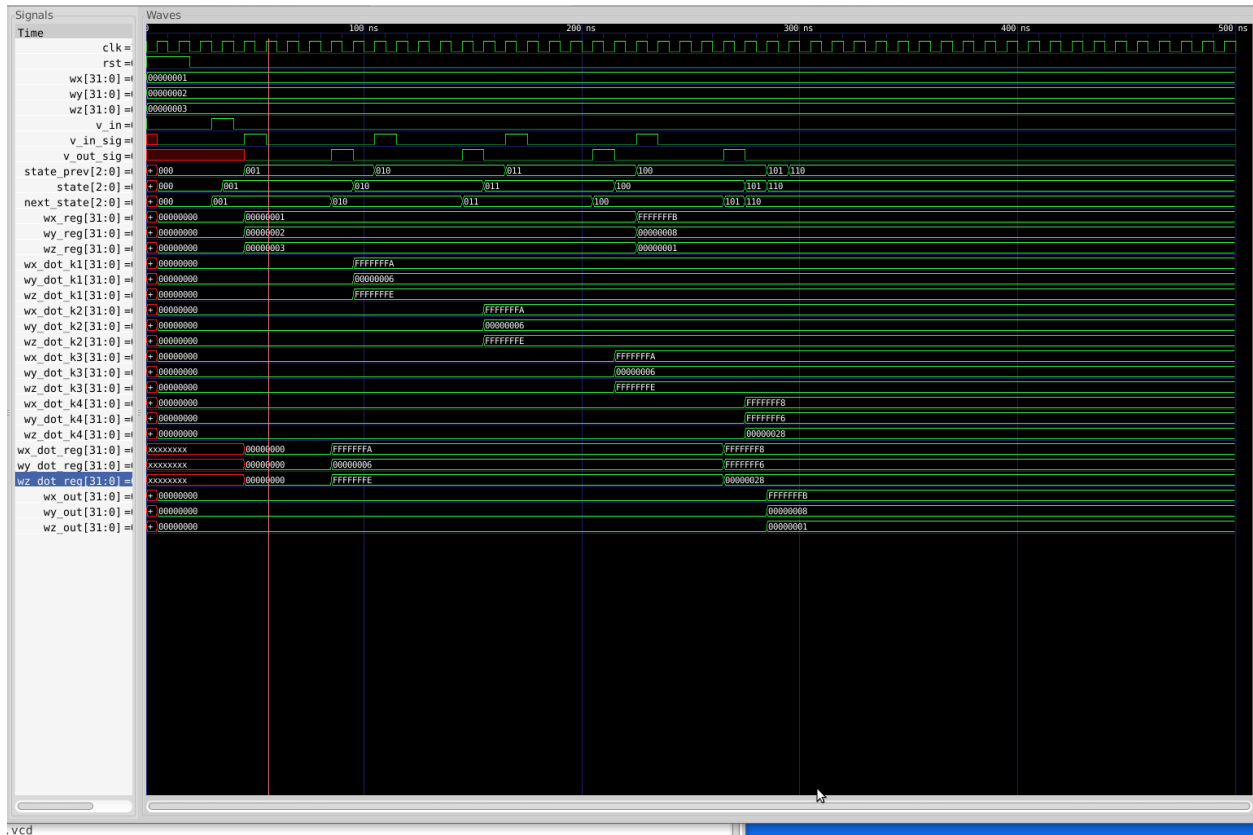


Figure 11: Euler rotational dynamics validation

Following validation of the Euler dynamics module, the full Runge-Kutta 4 (RK4) pipeline is verified using the Euler module as its computational core. In this test, sequential stages of the RK4 algorithm are exercised, with intermediate derivatives ( $k_1$  through  $k_4$ ) computed and accumulated across multiple clock cycles. The waveform demonstrates correct state progression through the pipeline, with each stage producing expected intermediate results and the final integrated outputs converging to the correct values. The alignment of valid signals and state registers confirms that the pipeline operates deterministically, with consistent latency from input to output. This validates both the correctness of the RK4 implementation and the integrity of its pipelined hardware structure. Further validation outside of simulation will be performed when synthesized and integrated.



## Next Steps

The next phase of development focuses on completing validation and integration across all system layers. While the SPI physical and protocol layers have been validated, the register layer requires further verification to ensure reliable storage and retrieval of data under all operating conditions. In parallel, the control FSM must be fully implemented, simulated, and validated to confirm correct sequencing between initialization, computation, and data availability. Once verified independently, the control FSM will be integrated with the numerical integration module, enabling coordinated execution of the simulation pipeline within the hardware-in-the-loop framework.

Following module-level validation, full system integration will be performed. This includes connecting the SPI interface, register map, control FSM, and numerical integration module into a unified design and validating end-to-end behavior. A key component of this process is comparison against a Python-based truth model of the rigid body dynamics. By applying identical inputs (initial conditions and torque profiles) and comparing outputs, the correctness of the hardware implementation can be quantitatively verified. The system currently operates using a 16-bit fixed-point representation (8.8 format), which provides a balance between numerical precision and hardware efficiency. Future work will include analyzing the impact of fixed-point quantization on simulation accuracy and determining whether additional scaling or precision adjustments are required.

## Significance

The development of an FPGA-based hardware-in-the-loop (HIL) simulation platform provides a critical advantage in the design of the attitude determination and control system (ADCS). Unlike software-based simulations running on microcontrollers or general-purpose processors, the FPGA implementation offers fully deterministic execution with effectively zero timing jitter in the simulation timestep. This guarantees that the numerical integration is performed at a constant  $\Delta t$ , eliminating timing variability that can otherwise introduce errors or instability in control system development and testing.

This deterministic timing capability also enables direct analysis and comparison of jitter effects when transitioning to the final ADCS implementation. By comparing the FPGA-based HIL results with those obtained from embedded execution of the same dynamics, it becomes possible to quantify the impact of timing jitter on system performance. This provides valuable insight into control robustness and allows for informed design decisions when selecting hardware platforms and control strategies. As such, the FPGA-based HIL system serves not only as a validation tool, but also as a foundational component in the iterative development and verification of the complete ADCS.